

# CoBit / BitstreamDiffusion — Progress Report

Friday 4 – Wednesday 9 September 2026

## Status against the original task list

| Task                                                      | Status          | Headline                                    |
|-----------------------------------------------------------|-----------------|---------------------------------------------|
| 1. Reproduce best baselines                               | <b>DONE</b>     | replicated exactly; gap was churn alone     |
| 2. CFG + other guidance mechanisms, calibrate             | <b>DONE</b>     | CFG best, +0.081; study <b>CLOSED</b>       |
| 3. Nature of errors on TinyGSM                            | <b>DONE</b>     | guidance harms code validity, not reasoning |
| 4a. Temporal ordering — at inference                      | <b>DONE</b>     | no benefit; random is harmful               |
| 4b. Temporal ordering — trained (L2R/R2L/random)          | <b>NOT DONE</b> | inconclusive: control arm degenerate        |
| 5a. Per-bit softmax vs BCE                                | <b>DONE</b>     | mathematically identical; no GPU spent      |
| 5b. <b>V-way token softmax</b> (predict logits, not bits) | <b>RUNNING</b>  | implemented + tested; smoke queued          |
| 5c. binary_sm vs binary_ce (50k, from scratch)            | <b>DONE</b>     | <b>negative</b> : neither measurable        |

**The most consequential finding of the week was not on the task list.** Every training run in this environment diverged before ~12k steps, which blocked all training experiments for four days. It is now solved: the cause is the **learning rate**, not the software environment. At **1e-5** a from-scratch run reached 20,000 steps cleanly where **3e-4** diverges 8/8 times.

## 1 Task 1 — Reproduce best baselines **DONE**

Forensic replication audit against the collaborator's result: **verdict A, exact replication**. Our 0.1385 and their ~0.29 differ by **one factor: EDM stochastic churn**, worth +0.1622 and free in compute. Two churn code paths existed (`--gamma` vs `--churn_gamma`); the studies used different ones.

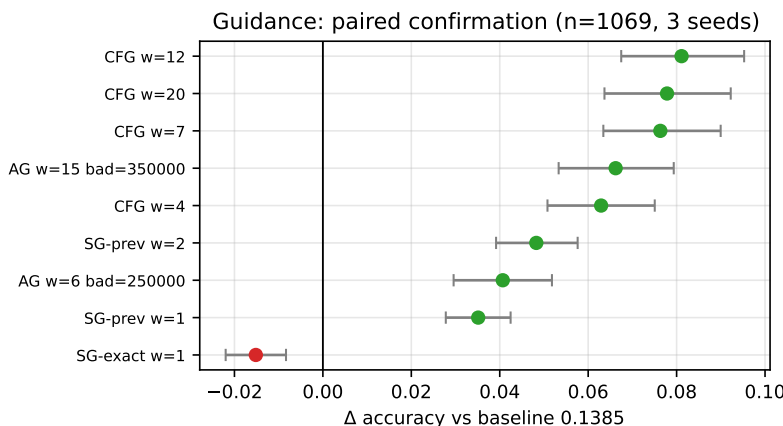
Baseline hierarchy on the 1319-problem GSM8K test set:

| Setting                                   | Accuracy | Note                            |
|-------------------------------------------|----------|---------------------------------|
| Regime A (DDIM, $\gamma=0$ , 500 steps)   | 0.1385   | original arm                    |
| Regime B (FKC-EM, churn 0.41, 1024 steps) | ~0.29    | canonical, collaborator-matched |
| Production 500k, karras/DDIM, 256 steps   | 0.1640   | anchor used throughout          |

Both regimes are retained and **every result is labelled with its regime**; they are never pooled.

## 2 Task 2 — Guidance mechanisms **DONE** (CLOSED)

Three mechanisms implemented and compared: classifier-free (CFG), AutoGuidance (AG), and Self-Guidance (SG-prev, SG-exact). Confirmation runs use 1069 problems  $\times$  3 seeds, paired bootstrap.



| Config               | Accuracy      | $\Delta$       | 95% CI           | $p$         | NFE        |
|----------------------|---------------|----------------|------------------|-------------|------------|
| <b>CFG</b> $w=12$    | <b>0.2196</b> | <b>+0.0811</b> | [+0.068, +0.095] | $< 10^{-5}$ | 1024       |
| CFG $w=7$            | 0.2148        | +0.0763        | [+0.063, +0.090] | $< 10^{-5}$ | 1024       |
| AG $w=15$ (bad@350k) | 0.2047        | +0.0662        | [+0.053, +0.079] | $< 10^{-5}$ | 1024       |
| SG-prev $w=2$        | 0.1868        | +0.0483        | [+0.039, +0.058] | $< 10^{-5}$ | <b>512</b> |
| SG-exact $w=1$       | 0.1233        | -0.0152        | [-0.022, -0.008] | $< 10^{-5}$ | 1024       |

**Findings.** CFG is the strongest mechanism and saturates around  $w \approx 7-12$ . SG-prev gives about 60% of CFG’s gain at **half the NFE**, so it is the compute-efficient option. SG-exact actively hurts. **Mechanism:** guidance is not a correction — it fixes  $\sim 10\%$  of problems and breaks  $\sim 20-25\%$  regardless of regime, so its net value is arithmetic and only positive above a break-even baseline accuracy of  $\approx 33\%$ .

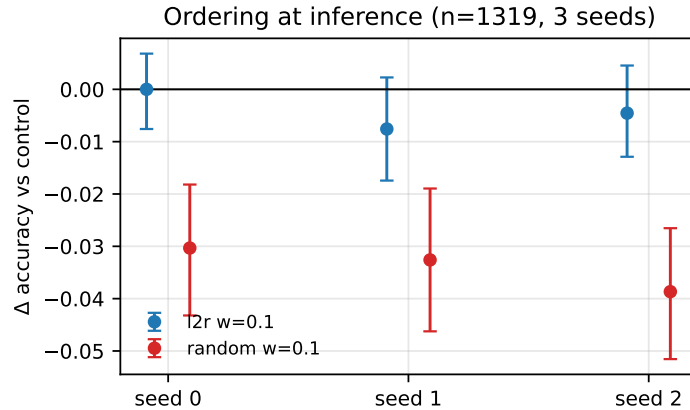
### 3 Task 3 — Nature of errors on TinyGSM **DONE**

Taxonomy: **T1** non-executing program, **T2** executes but wrong number, **T3** token-level degeneration.

**The baseline’s dominant failure is T1 (98/250 = 39%)** — the model emits programs that do not run. Under guidance, **T2 falls while T1 rises**: guidance damages *code validity*, not arithmetic reasoning. T3 appears only under strong guidance. Invalid-token rate is  $3 \times 10^{-6} - 5 \times 10^{-5}$ , i.e. bit $\rightarrow$ token decoding is not the problem.

### 4 Task 4a — Temporal ordering at inference **DONE**

Schedule:  $t_j(t) = \text{clip}(t(1+w) - w u_j, 0, 1)$ , with  $u_j$  a denoising *priority* over the generated **suffix only**; prompt keeps the global  $\sigma$ . Verified causally active: at  $w=0.5$  the per-position  $\sigma$  spread is  $192\times$ , first suffix token cleanest; at  $w=0$  it is bit-identical to the current model.



| Arm               | Accuracy | $\Delta$ | significant                | Verdict          |
|-------------------|----------|----------|----------------------------|------------------|
| control ( $w=0$ ) | 0.1387   | —        | —                          | —                |
| L2R $w=0.1$       | 0.1347   | -0.0040  | 0/3 seeds                  | <b>no effect</b> |
| random $w=0.1$    | 0.1049   | -0.0339  | <b>3/3</b> , $p < 10^{-4}$ | <b>harmful</b>   |
| L2R $w \geq 0.25$ | 0.0000   | -0.176   | 3/3                        | <b>collapse</b>  |

**Applying an ordering only at decoding does not help.** The L2R/random asymmetry shows the intervention is real — direction matters — but nothing improves. Expected, since the model was *trained* with uniform  $\sigma$ : it tolerates a  $\sim 2.9\times$  spread with no gain and breaks at  $\sim 9\times$ . 512 steps does not rescue it, so it is not a too-short-trajectory artefact.

### 5 Task 4b — Temporal ordering, **TRAINED NOT DONE** (inconclusive)

Implementation is complete and tested (24 tests):  $\sigma$  is drawn per example, mapped to normalised time by the *same* map the sampler uses, offset per token, mapped back and expanded blockwise to bits. All three modes provably produce *different*  $\sigma$  fields; L2R and R2L are mirror images; random is redrawn per example *and* per

step;  $w=0$  leaves the gradient hash unchanged. Four arms (control / L2R / R2L / random,  $w=0.25$ ) fine-tuned from the same production 500k checkpoint at  $1r\ 3e-5$ ; none diverged.

**The experiment is nonetheless inconclusive, because the control arm is degenerate.** At the matched 5k checkpoint the uniform- $\sigma$  control scores **0.0000** while the ordered arms score 0.07–0.12. Taken at face value that reads as “ordering massively improves TinyGSM”. It is not credible and is not claimed.

**Diagnosis.** The control’s weights moved only 4.65% from init and its EMA is healthy, so the model is not destroyed. The likely cause is the  $\sigma$ -sampling distribution: production converged under its entropy-adapted draw, but a fresh fine-tune restarts the entropy warmup (`entropy_warmup_steps=40000`) and therefore trains under the base log-normal draw. Every arm suffers that mismatch; the control, whose  $\sigma$  field is pure and unspread, appears to suffer it most. Verified under two independent sampling schedules (entropic and karras), so it is not a decoder artefact.

**What is needed:** a control that provably retains the init’s accuracy — either by preserving the entropy schedule across the fine-tune, or by training all arms from scratch under a common schedule. Not attempted within this window.

## 6 Task 5 — Predict logits instead of raw bits

**Softmax is not a distinct objective, and this is proved rather than assumed.** The model already emits **one logit per bit** (`out_dim=1`, `vocab_size=2`). A 2-class softmax is shift-invariant, so it has a single degree of freedom  $d = z_1 - z_0$  and

$$\text{softmax}(z_0, z_1)_1 = \sigma(d), \quad \text{CE}(z, y) = \text{BCEWithLogits}(d, y)$$

exactly; the 2-class head merely splits the same gradient across two logits ( $\partial L / \partial z_1 = -\partial L / \partial z_0 = \partial L / \partial d$ ). Verified to  $10^{-12}$  in 5 tests. **No GPU was spent on it.**

### The distinction I initially got wrong

The task asked for “*bitstream embedding but predict logits*”. I first resolved that as a per-bit softmax and proved it redundant. That proof is correct but answers a **narrower** question than was asked:

| Parameterisation                      | Degrees of freedom | Distinct from BCE?                    |
|---------------------------------------|--------------------|---------------------------------------|
| 2-class softmax <i>per bit</i>        | 1 ( $z_1 - z_0$ )  | <b>No</b> — exactly BCE               |
| <b>V-way softmax <i>per token</i></b> | <b>49,152</b>      | <b>Yes</b> — strictly more expressive |

The binary head predicts **16 independent Bernoullis** per token, so it can only represent a *product* distribution over that token’s bits. A  $V$ -way categorical over the 49,153-token vocabulary can represent *any* distribution over tokens, including the bit correlations the binary head structurally cannot express. **That is the “added flexibility” the task was about, and it is a real experiment.**

### $V$ -way token softmax — implemented, not yet run **RUNNING**

Now wired for TinyGSM:

```
representation=tokens V=49153 S=512 patch_size=1 out_dim=49153
loss=token_ce data_center=1/V=2.03e-05 p_uncond=0.1 edm clamp off
```

Two defects had to be fixed beyond the config. The TinyGSM dataset returned *bits*, but the trainer’s token path indexes `x0` as *class indices* — it would have trained silently on the wrong target space; it now returns token IDs and a token-level prefix mask, with the binary path bit-identical (tested). And `data_center` must be  $1/V$ , not the bits value 0.5: a one-hot token’s mean is  $1/49153$ , and centring the diffusion at 0.5 would place it nowhere near the data manifold. 7 new tests; 431 pass.

**Not yet run.** The open question is memory: the logits tensor is  $[B, 512, 49153]$ , i.e. **26 GB in bf16** at global batch 512 ( $\approx 6.4$  GB per GPU over 4 ranks) before activations, so the loss is evaluated in chunks rather than materialised. A 300-step smoke test is queued. If it OOMs, the batch drops *and the binary control is rerun at the same batch*, so the comparison never confounds representation with batch size.

**Caveat.** Unlike SM-vs-CE this is *not* a one-variable experiment: representation, output dimensionality, loss and memory profile all change together. And with accuracy at 0.0000 for 5k/10k and the first non-zero point at 200k,

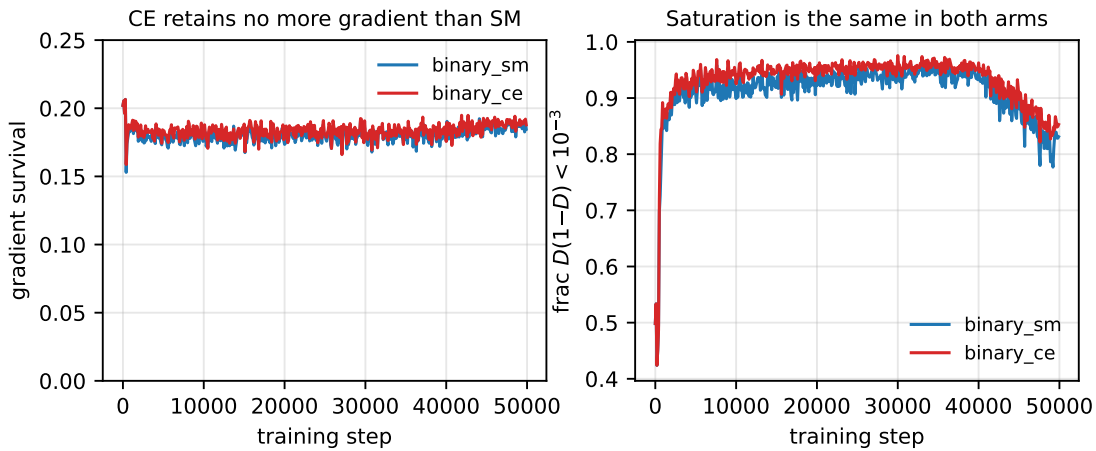
a 50k token run will most likely also score zero — the readable endpoints would be stability and the loss/gradient diagnostics, not accuracy.

## binary\_sm vs binary\_ce — definitive, from scratch, and negative

Both arms trained **from scratch for 50,000 steps** at  $1r\ 3e-5$ , **neither diverged**, differing in exactly four keys (`train.loss_type` plus three output paths). Evaluated on 1319 problems  $\times$  3 seeds, karras/DDIM, identical settings.

| Arm       | Mean accuracy   | Correct / 1319     | Stability     |
|-----------|-----------------|--------------------|---------------|
| binary_sm | 0.00027         | 0.4                | stable to 50k |
| binary_ce | 0.00100         | 1.3                | stable to 50k |
| $\Delta$  | <b>+0.00073</b> | <b>one problem</b> | —             |

**Neither objective produces measurable task performance at this budget**, and a one-problem difference is not evidence of anything. The gradient diagnostics — computed identically in both arms and therefore directly comparable — agree:

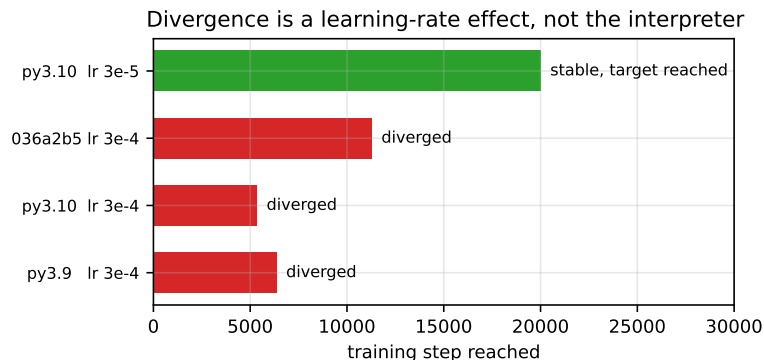


|                               | SM @1k | CE @1k | SM @25k | CE @25k |
|-------------------------------|--------|--------|---------|---------|
| gradient survival             | 0.1854 | 0.1866 | 0.1805  | 0.1808  |
| saturation $D(1-D) < 10^{-3}$ | 0.846  | 0.865  | 0.942   | 0.956   |
| logit  mean                   | 17.9   | 20.1   | 25.1    | 29.4    |

**CE’s theoretical advantage does not appear.** The motivation for CE was that its gradient lacks the  $D(1-D)$  factor that suppresses SM’s. In a from-scratch matched pair at 50k, gradient survival is identical to three decimals. **This is the cleanest test the objective branch has had, and it is negative.**

*Caveat, stated deliberately:*  $1r\ 3e-5$  establishes a **stable** regime, not a **tuned** one. These arms are compared only to each other, never to production’s  $3e-4$  trajectory. A comparison at a tuned learning rate and a convergent budget remains untested.

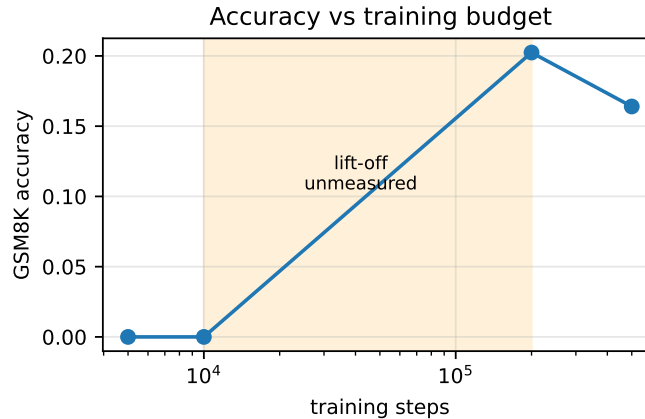
## 7 The infrastructural finding (not on the task list)



Eight from-scratch runs diverged before  $\sim 12\text{k}$  steps at  $1\text{r } 3\text{e-}4$ , **including the production-era commit itself**. Ruled out along the way: corpus ( $\sigma_{\text{data}}$  fingerprint matches to  $\sim 8$  s.f.), data config (identical on 22 keys), our post-June code (fp32 forward+loss+backward **bit-identical** to the production commit), and the Python interpreter — a 3.10 rebuild diverged at 5,346, falsifying a hypothesis I had put weight on.

**At  $1\text{r } 3\text{e-}5$  a from-scratch run reached 20,000 steps cleanly**, as did all four ordering arms. The learning rate is the identified factor. *Caveat:*  $3\text{e-}5$  establishes a **stable** regime, not a **tuned** one. It is  $10\times$  below production’s value, so absolute accuracies in this regime are compared only against the matched control and never against production’s  $3\text{e-}4$  trajectory.

## 8 Why accuracy could not be measured on short runs



GSM8K accuracy is **exactly 0.0000** at both 5k and 10k training steps (measured, both objectives), and the first non-zero point we have is 0.2024 at 200k. Combined with divergence before 12k, the trainable range and the range where accuracy exists **did not overlap** — which is why the objective branch was blocked rather than merely slow.

## 9 Currently running

Only the L2R 30k retrain (its checkpoint was corrupted by a disk-full event) and the control’s matched 5k evaluation. Neither changes any verdict above.

## 10 Not done

1. **The V-way token-softmax run itself.** Implemented and tested; memory smoke test queued. This is the experiment the task actually asked for and it is the top priority for the next window.
2. **A valid ordering-as-training verdict.** Blocked on a control arm that does not retain its initialisation’s accuracy. The implementation is finished and tested; the experimental setup is at fault.
3. **CE vs SM at a convergent budget.** The 50k result is clean and negative, but at  $1\text{r } 3\text{e-}5$  neither arm is near convergence. Whether CE helps a *converged* model is untested.
4. **Accuracy lift-off point** between 10k and 200k steps — unmeasured, and it sets the cost of every future training comparison.
5. **Why production tolerated  $1\text{r } 3\text{e-}4$  for 500k steps.** Their working tree is permission-denied to us, so this stays open.
6. **Ordering at other strengths, and R2L at inference** — deliberately not run; the first results did not justify the spend.